INFORME TÉCNICO

Desarrollo del proyecto Ciudad 2050

Recorrido 40 | Juego educativo en 3D y realidad virtual (VR)

Autor: Matías Ortiz

26 de octubre de 2026

DE QUÉ TRATA

Este informe explica cómo desarrollé una experiencia escolar interactiva: qué lenguajes y herramientas utilicé, cómo transformé referencias reales en un entorno recorrible, cómo organicé la historia y las interacciones, y qué decisiones tomé para llevar el proyecto a la computadora, el celular y la realidad virtual.

TECNOLOGÍA PRINCIPAL	EXPERIENCIA
TypeScript · Babylon.js · Vite	Navegador · WebXR · escritorio y móvil

Documento elaborado a partir de la estructura, la configuración y la documentación técnica del proyecto.

Resumen del proyecto

Desarrollé Ciudad 2050 como una experiencia 3D que evolucionó hasta convertirse en Recorrido 40, un juego educativo centrado en la Escuela CIMDIP & Miguel Cané. El jugador puede desplazarse por el barrio y por distintos espacios de la escuela, conversar con personajes, completar actividades y seguir una historia vinculada con el aniversario institucional.

La aplicación se ejecuta en el navegador. En una computadora se controla con teclado y mouse; en pantallas táctiles adapta la interfaz al celular; y en un visor compatible puede iniciar una sesión inmersiva mediante WebXR. La escuela funciona como escenario principal y el entorno urbano de Ciudad 2050 le da contexto.

La decisión técnica central fue construir el mundo directamente con código y Babylon.js. Esto permite controlar la escala, la disposición y el comportamiento de los elementos, y mantener una ruta web común para escritorio y realidad virtual.

Origen y objetivo

El proyecto comenzó como una propuesta de ciudad solarpunk para el año 2050 y evolucionó hacia una experiencia escolar: representar la Escuela CIMDIP & Miguel Cané y sus alrededores, y convertir el recorrido en una actividad interactiva. La historia de Recorrido 40 organiza esa exploración alrededor del aniversario de la institución.

El objetivo técnico y educativo fue combinar un espacio reconocible con una experiencia que invite a explorar. Por eso, el jugador puede observar el edificio, recorrer interiores, recibir orientación de personajes y responder preguntas integradas en la historia.

Lenguajes y herramientas

TypeScript es el lenguaje principal del código. Añade tipos y comprobaciones al proceso de desarrollo; Vite transforma y organiza esos módulos para que el navegador ejecute JavaScript. La configuración activa el modo estricto de TypeScript, que ayuda a detectar inconsistencias antes de ejecutar el juego.

Tecnología	Función en el proyecto
TypeScript	Lógica del juego, construcción de escenas, controles, interacciones, audio y sistemas de VR.
Babylon.js 8	Motor 3D: cámaras, geometría, materiales, iluminación, renderizado y herramientas WebXR.
HTML y CSS	Documento base, pantalla de carga, HUD, menús, botones y presentación de la interfaz.
Vite y npm	Servidor local, resolución de módulos, administración de dependencias y compilación de producción.
WebXR	Estándar del navegador que permite solicitar sesiones inmersivas y recibir datos de los controles del visor.
Web Audio API	Síntesis de sonidos y ambientes desde código, sin depender de archivos de audio para los efectos descritos.
Vitest, ESLint y Prettier	Pruebas automatizadas, análisis estático y formato consistente del código.
Puppeteer e iwer	Automatización de verificaciones en navegador y simulación de controles de realidad virtual.

Cómo pasé de las referencias al escenario

Para definir la escuela, utilicé el plano de evacuación de la planta baja como referencia de distribución y proporciones. En el código, ese plano se escaló a razón de 0,07 metros por píxel. Las vistas públicas de Street View, el recorrido virtual de 2020 y el material audiovisual más reciente aportaron referencias para las fachadas, los patios, las aulas, el gimnasio, la señalización y el mobiliario.

Después convertí esas referencias en datos de construcción: posiciones, medidas, niveles, materiales, puertas, pasillos y zonas transitables. La planta baja sigue la distribución del plano; los niveles superiores y el jardín se completaron con las referencias disponibles. Cuando una referencia no describe un ambiente interior, el modelo evita atribuirle una distribución que no está documentada.

El resultado es una reconstrucción tridimensional interpretativa hecha con geometría y reglas programadas. No es un escaneo fotogramétrico ni una réplica milimétrica obtenida automáticamente de Google Maps.

Construcción procedimental del mundo

La mayor parte del escenario se genera al iniciar el juego. En lugar de colocar cada pieza como un objeto independiente, el código reúne elementos repetidos, como ventanas, baldosas, árboles y luminarias, y calcula sus transformaciones. Los edificios, las calles, la vegetación y parte del mobiliario se forman combinando primitivas 3D.

Las texturas y los carteles se dibujan en un canvas del navegador, es decir, en un lienzo HTML controlado desde el programa. Así se generan superficies como hormigón, madera, césped, metal y señalización sin cargar un catálogo de imágenes externas. Los materiales y la iluminación se organizan en módulos compartidos para facilitar los cambios visuales y evitar duplicar lógica.

El barrio se genera a partir de una semilla. Al repetirla, el entorno conserva la misma distribución, lo que facilita depurar y comparar cambios. La estructura de la escuela permanece estable y ocupa el centro de la escena.

Organización del código

Separé el proyecto por responsabilidades. El archivo principal conecta los sistemas; los módulos del mundo construyen la escuela y el barrio; y el juego, el jugador, la interfaz, el audio y la realidad virtual se mantienen en áreas distintas. Esta división permite modificar una parte sin tener que concentrar toda la lógica en un solo archivo.

Módulo	Responsabilidad
src/main.ts	Inicializa el motor, crea la escena y conecta controles, mundo, calidad y director del juego.
src/world/	Escuela, ciudad, edificios, materiales, iluminación, población y elementos ambientales.
src/core/	Representación por instancias, calidad gráfica y canal de renderizado.
src/player/	Cámara, movimiento, gravedad, colisiones y entrada virtual.
src/game/	Historia, personajes, diálogos, objetivos, actividades, guardado e interacciones.
src/vr/	Inicio de la sesión WebXR, mandos, puntero láser, locomoción y ayudas de confort.
src/ui/ y src/audio/	HUD (información superpuesta en pantalla), minimapa, controles táctiles, sonido ambiental y efectos.

Historia, personajes e interacción

La experiencia se organiza como Recorrido 40. Un prólogo y cinco capítulos guían al jugador por distintos sectores. Diez personajes con nombre presentan objetivos y acompañan el recorrido; las conversaciones y preguntas permiten que el jugador participe en lugar de limitarse a observar la escena.

El juego incluye seis actividades principales: una ruta con el robot Educabot, una propuesta de alimentación saludable, un desafío de reciclaje, una trivia sobre trofeos, una secuencia de baile y una ronda de penales. También incorpora un simulacro de evacuación, una búsqueda de estrellas en el jardín y un acto final. El Pasaporte 40 registra ocho sellos y el progreso se guarda localmente en el navegador.

En computadora, la interacción combina proximidad, mirada y teclado. Al acercarse a un personaje u objeto se puede activar la acción correspondiente. En el visor, un rayo desde el control selecciona paneles y elementos; los botones del mando confirman, vuelven o abren el menú.

Controles y realidad virtual

La versión de escritorio ofrece movimiento con WASD, carrera, salto e interacción con teclado. En celular, la interfaz pasa a controles táctiles y joysticks en pantalla. Para VR, el stick izquierdo permite caminar, el derecho controla el giro o el desplazamiento alternativo y los gatillos permiten apuntar y seleccionar. Los paneles de juego y el indicador de objetivos se adaptan al espacio del visor.

WebXR requiere que el navegador y el visor admitan el modo inmersivo y que el sitio se abra en un contexto seguro. Durante el desarrollo local, el servidor de VR se ejecuta con HTTPS. La implementación agrega opciones de confort, como giro por pasos, viñeta, velocidad ajustable y modo sentado, para que cada persona pueda elegir una forma de desplazamiento más cómoda.

La experiencia VR comparte el mundo y la lógica del juego con la versión web; cambia la forma de recibir la mirada, el movimiento y las acciones del usuario.

Rendimiento, carga y compatibilidad

El rendimiento se trabajó desde la representación de la escena. Babylon.js permite usar instancias finas (thin instances) para dibujar muchas copias de una misma malla con menos objetos de gestión y menos llamadas de dibujo. Un índice espacial propio resuelve consultas de colisión, altura del piso e inspección sin convertir cada elemento repetido en una entidad independiente.

- Los perfiles de calidad ajustan el detalle, la densidad de vegetación, los efectos visuales y la población para responder a equipos diferentes.
- La funcionalidad WebXR y algunos controles se importan de forma diferida cuando hacen falta; la carga normal de escritorio evita descargar de entrada módulos destinados al visor.
- La preparación de los sombreadores gráficos (shaders) tiene un límite de tiempo durante la carga. Si un equipo necesita más tiempo, el navegador puede completar parte del trabajo cuando esos materiales se utilizan.
- La aplicación incluye una configuración de aplicación web progresiva (PWA), que permite instalar el juego y guardar recursos en caché para visitas posteriores cuando el navegador lo admite.

Estas medidas reducen trabajo y carga, pero el rendimiento final depende de la GPU, el navegador, la resolución y el visor utilizados. Las mediciones de escritorio no representan por sí solas los fotogramas por segundo de una sesión VR.

Pruebas y revisión técnica

El repositorio tiene comandos separados para comprobar tipos, aplicar reglas de estilo, ejecutar pruebas unitarias y generar la versión de producción. Las pruebas unitarias cubren lógica como la generación del plano, el generador de números aleatorios, el índice espacial, la historia, los controles y ciertos sistemas de población. Las herramientas de navegador permiten observar la escena, probar interacciones y medir la carga.

Comando	Uso
npm run dev	Iniciar la versión de desarrollo en el navegador.
npm run dev:vr	Iniciar el servidor HTTPS de desarrollo para probar WebXR desde la red local.
npm run typecheck	Revisar tipos de TypeScript sin emitir archivos.
npm run lint	Detectar problemas de código con ESLint.
npm test	Ejecutar el conjunto de pruebas unitarias con Vitest.
npm run build	Verificar el proyecto y compilar la versión de producción.

La simulación de controles de VR y las pruebas automatizadas ayudan a encontrar errores antes de una demostración. Para cerrar una validación de VR, todavía es importante comprobar la experiencia en el visor objetivo y en su navegador, porque la compatibilidad y el rendimiento varían entre dispositivos.

Proceso de desarrollo

Organicé el trabajo de manera iterativa. Primero definí el lugar, las referencias y la escala. Luego construí el escenario y la navegación; después incorporé la historia, los personajes y las actividades. A continuación adapté los controles a escritorio, celular y visor, y continué corrigiendo problemas visuales, de carga y de interacción.

Utilicé Codex como apoyo de programación para analizar el código, explorar soluciones y depurar errores. Yo definí los objetivos y seleccioné las referencias; también revisé las propuestas y decidí qué cambios integrar para que el proyecto conservara su propósito.

El resultado es una aplicación web 3D modular cuya escuela, barrio, personajes, actividades y controles se definen desde el código. La elección de TypeScript y Babylon.js permitió conservar una base común para el navegador y WebXR, mientras que la organización por módulos hizo posible ampliar el contenido por etapas.

Uso profesional de la inteligencia artificial y otros proyectos

Trabajo profesionalmente con herramientas de inteligencia artificial y las integro en distintas etapas de mis proyectos: investigación, organización de ideas, creación de prototipos, programación, análisis de errores y revisión. Las utilizo para acelerar tareas y explorar alternativas; yo establezco los objetivos, evalúo sus propuestas y verifico que los resultados sean útiles y correctos.

Ciudad 2050 es uno de los muchos proyectos que desarrollo. También trabajo en otras iniciativas tecnológicas y creativas, con objetivos y herramientas diferentes. Cada proyecto me permite aplicar lo que aprendo y seguir ampliando mi experiencia.

Términos técnicos

Término	Explicación sencilla
Motor 3D	Software que calcula y muestra la escena, la cámara, la luz y los materiales.
Procedimental	Contenido creado mediante reglas programadas cuando se ejecuta el juego.
Instancia fina (thin instance)	Copia de una malla compartida cuya posición, tamaño y orientación se guardan como datos.
WebXR	Estándar del navegador para acceder a sesiones de realidad virtual o aumentada compatibles.
Compilación	Proceso que prepara módulos y recursos para publicar y ejecutar la aplicación.

Mapa de archivos principales

La estructura técnica se puede consultar directamente en el repositorio. Los archivos de entrada más útiles para entender el desarrollo son:

- package.json y tsconfig.json: dependencias, comandos y reglas del compilador.
- src/main.ts: inicio y conexión de los sistemas.
- src/world/SchoolLayout.ts y src/world/SchoolBase.ts: distribución y geometría de la escuela.
- src/world/people/: apariencia, movimiento y comportamiento de estudiantes y personajes.
- src/game/GameDirector.ts y src/game/story/: dirección del recorrido, historia, objetivos y guardado.
- src/core/QualityManager.ts y src/core/InstanceFarm.ts: calidad gráfica e instanciado.
- src/vr/: integración del visor, mandos, movimiento y confort.
- tests/ y tools/: pruebas unitarias y herramientas de verificación en navegador.

Conclusión

Construí el proyecto combinando programación web, gráficos 3D y diseño de interacciones. TypeScript sostiene la lógica y la organización; Babylon.js renderiza el mundo; Vite prepara la aplicación; y WebXR conecta la experiencia con visores compatibles. Las referencias reales dieron forma al escenario, mientras que la geometría generada por código, los sistemas de juego y las pruebas permitieron convertirlo en un recorrido interactivo que puede seguir evolucionando.